Aditya Dhiman

778-302-4429 | aditya16dhiman@gmail.com | https://www.linkedin.com/in/adityadhiman18/ | aditya-dhiman4.github.io

Skills

Languages: C, C++, Python, MATLAB, ARM Assembly, SystemVerilog

Embedded: FreeRTOS, STM32, ARM Cortex-M, Embedded Linux, Yocto, BitBake, ROS2

Tools: Vector CANoe, JTAG/SWD, Logic Analyzer, Simulink, Bazel, CMake, GoogleTest, gRPC, Protobuf, Docker, Altium

Education

University of British Columbia — *BASc, Computer Engineering* *Expected May 2028*

Work Experience

Autopilot Embedded Software Engineer Intern *Palo Alto, CA*
Tesla *June 2026 – Sept 2026*

- Developed firmware in **C** for Tesla's **AI4** Autopilot computer, root-causing **watchdog**, **data abort**, and **power fault** resets via fleet **CAN** and on-target logs; fixes shipped to **millions of vehicles** with **2x** fleet MCU stability
- Resolved the top-ranked reset cause, power faults mis-attributed as **data aborts**: gated a DDR error **ISR** that aborted before the pending reset and re-attributed resets to **PGOOD**, routing them to the hardware team
- Eliminated a vehicle program's top reset cause, a **data abort** in a memory-timeout ISR; found with the **silicon vendor** that competing bus traffic starved its error-register reads and shipped a stopgap cutting occurrences to **zero**
- Designed an **I2C bus force-reclaim** that releases the orphaned mutex and clocks **SCL** so reset reasons log from **exception context** when a reset preempts a transaction; fixed log loss seen at nearly every branch sign-off
- Extended a **Python/Jupyter** pipeline ranking reset causes across the full production fleet; built a bootloop detector from power state machine **CAN** signals and filters for known issues, exposing the true top firmware faults
- Built **HIL** fault-injection tests (forced I2C bus lock-ups, resets mid-transaction) that reproduce latent firmware faults and run at every branch sign-off to catch regressions

Software Engineer Intern – Infotainment Platform *Vancouver, BC*
Rivian and Volkswagen Group Technologies *Sept 2025 – April 2026*

- Developed an Ethernet switch manager for the Rivian R2 in **C++**, reading link status, temperature, and heartbeat registers and streaming diagnostics to OBD-II clients over a multithreaded **gRPC/Protobuf** server
- Architected a **crash-resilient** logging framework in C++ with persistent **UFS** storage, automatic log rotation, and cloud upload across ECU power-cycles, reducing post-crash debug time across Infotainment and Connectivity organizations
- Developed a **GPT partitioning tool** for embedded Linux images integrated into flashing scripts and **CI Pipelines**, replacing a flaky legacy flow and enabling faster, more reliable infotainment ECU flashing for **200+ developers**
- Maintained **Embedded Linux** builds using **Yocto/BitBake**, authoring and debugging recipes for application and BSP layers across current and next-generation infotainment ECUs

Design Team Experience

Vehicle Controls Lead *Sept 2024 – Present*
UBC Formula Electric (FSAE)

- Leading a team of 4 architecting quad-motor **torque vectoring** in **C++**, using **nonlinear optimization** to allocate wheel torques for desired force and yaw moment under combined-slip tire dynamics; **2.7 s** 0–100 km/h in simulation
- Developed real-time firmware for custom **STM32H5/H7** ECUs in **C++** with **FreeRTOS**, **CAN/I2C/SPI/UART**, and ADC/GPIO/PWM; unit-tested with **gTest** and validated bring-up with **SWD** and logic analyzers
- Wrote an automotive-grade **IMU** driver over **SPI** with on-chip filtering and FIFO buffering, supplying acceleration and angular rate to the yaw rate controller and EKF for real-time vehicle dynamics control
- Implemented a fixed-point **CORDIC** driver, cutting trigonometric CPU cycles 3× and torque vectoring loop latency
- Designed a fault-tolerant, hardware-agnostic **eFuse** driver shared across ECUs, with **DMA**-driven current sensing, fault handling, and retry logic, improving vehicle start-up and fault-state reliability
- Integrated **PID** yaw rate control, per-wheel **PID** speed control, and **battery-aware power limiting**; built a **Simulink** driver simulation for closed-loop testing and validated yaw control on vehicle
- Built a **CAN-FD**/Classic CAN driver supporting **64-byte** frames, enabling legacy and next-gen ECU communication
- Engineered an **Extended Kalman Filter** in **C++** fusing IMU, wheel speed, and GPS, with outlier rejection for wheel slip and GPS dropouts, for body velocity estimation below 10 km/h